

# Apigee X Simulator

## Build-Your-Own-Proxy Exercises (with Guidance)

Unlike the previous two documents (which tested proxies that already exist in the seed data), every exercise here has you **create something new** from a blank proxy. Try each one yourself first using just the "Your task" box; the "Guided solution" underneath is there if you get stuck or want to check your work afterward - don't read ahead if you want the practice to count.

Login: admin@apigee-sim.local / Apigee123!. Start every exercise from Proxies > + Create Proxy.

### Exercise 1 - A protected read-only proxy

*Practices: proxy creation, VerifyAPIKey, SpikeArrest, basic Trace verification*

**Your task:** Create a new proxy called `InventoryService-v1`, base path `/inventory`, using the `ProductService` target server. Protect it with an API key AND limit it to 10 requests/second.

#### Guided solution

- 1 Proxies > + Create Proxy. Name: `InventoryService-v1`. Base Path: `/inventory`. Initial Target URL: paste `ProductService`'s host (Target Servers page > copy the host for `ProductService`), or leave the default and fix it in step 3.
- 2 Open the new proxy > Develop tab > Target Endpoints panel > set Target Server = `ProductService`.
- 3 Policies panel > Add > `VerifyAPIKey` > `apiKeyLocation=header`, `apiKeyParam=x-api-key` > Save (attaches to PreFlow by default).
- 4 Add another policy > `SpikeArrest` > `rate=10ps` > Save.
- 5 Deploy tab > Deploy to test.

#### Verify:

GET `/inventory` Header: `x-api-key: <any Developer App's Consumer Key>`  
Expected: `VerifyAPIKey = success`, `SpikeArrest = success`, 200 response with product data.  
Repeat 11+ times quickly -> `SpikeArrest` should fail with 429 on the 11th+ call.  
Remove the header entirely -> 401 "Failed to resolve API Key variable..."

### Exercise 2 - Multiple HTTP methods, multiple target endpoints (Route Rules)

*Practices: exactly what you originally asked for - one proxy, several verbs, each routed to a different backend*

**Your task:** Create `WarehouseService-v1`, base path `/warehouse`. GET requests must go to the `CustomerService` target server; POST, PUT, PATCH, and DELETE must all go to the `OrderService` target server instead - on the SAME proxy.

#### Guided solution

- 1 Create the proxy as usual (the Initial Target URL doesn't matter much - you'll replace it).
- 2 Develop tab > Target Endpoints > rename/repoint the existing "default" endpoint's Target Server to `CustomerService`.
- 3 Click "Add Target Endpoint", name it `write-target` (when prompted), set its Target Server to `OrderService`.
- 4 Route Rules panel > the existing default rule should point at your `CustomerService` endpoint - if not, add a rule: Verb=GET, Target Endpoint=.
- 5 Add a second Route Rule with NO verb set (blank = matches everything else) and Target Endpoint=`write-target`. Make sure this one is listed AFTER the GET rule (use the up/down arrows).
- 6 Deploy to test.

#### Verify:

|                                          |                                                                               |
|------------------------------------------|-------------------------------------------------------------------------------|
| GET <code>/warehouse</code>              | -> Routing step names the <code>CustomerService</code> -backed endpoint       |
| POST <code>/warehouse</code> (body: {})  | -> Routing step names <code>write-target</code> ( <code>OrderService</code> ) |
| PATCH <code>/warehouse</code> (body: {}) | -> also routes to <code>write-target</code>                                   |

DELETE /warehouse

-> also routes to write-target

## Exercise 3 - Conditional Flow for one specific sub-resource

*Practices: conditional flows, path-suffix conditions, attaching a policy to a specific flow*

**Your task:** On WarehouseService-v1 from Exercise 2, add a Conditional Flow that ONLY applies to GET /warehouse/reports and adds a response header X-Report-Mode: true - every other GET request should be unaffected.

### Guided solution

- 1 Develop tab > Conditional Flows > Add Conditional Flow. Set Verb=GET, Base Path Suffix=/reports.
- 2 Policies panel > "Attach new policy to" > pick the new flow's (response) option > Add > AssignMessage > assignTo=response, setHeaders: X-Report-Mode: true > Save.

### Verify:

GET /warehouse/reports -> timeline shows a step under "Conditional Flow: <name> (Response)"; response headers include X-Report-Mode: true  
GET /warehouse -> no such step runs, no extra header

## Exercise 4 - Your own OAuth2-protected proxy pair

*Practices: OAuthV2 GenerateAccessToken + VerifyAccessToken end-to-end, using two cooperating proxies (matches the pattern of OAuthToken-v1 / OAuthProtected-v1, but built by you)*

**Your task:** Build two proxies: TokenIssuer-v1 (base path /issue-token) that issues an OAuth token, and Vault-v1 (base path /vault) that requires a valid token, read from the query parameter token instead of the Authorization header.

### Guided solution

- 1 Create TokenIssuer-v1 (target server doesn't matter, e.g. CustomerService). Add policy OAuthV2: operation=GenerateAccessToken, grantType=client\_credentials, scopes=write.
- 2 Create Vault-v1 (target server CustomerService). Add policy OAuthV2: operation=VerifyAccessToken, Access Token Location=Query Parameter, Query Param Name=token.
- 3 Deploy both to test.

### Verify:

POST /issue-token Body: {"client\_id":"<key>","client\_secret":"<secret>"}  
-> 200, response body contains access\_token

GET /vault?token=<access\_token from above>  
-> OAuthV2 VerifyAccessToken = success

GET /vault (no token param at all)  
-> 401 "Invalid Access Token", message names the query param it expected

## Exercise 5 - JWT with your own issuer/audience

*Practices: VerifyJWT configuration and constructing a matching token by hand*

**Your task:** Create PartnerAPI-v1, base path /partner, target server OrderService. Require a JWT with issuer `https://partner.example.com` and audience `partner-api`, algorithm HS256.

### Guided solution

- 1 Create the proxy, add policy VerifyJWT: algorithm=HS256, issuer=https://partner.example.com, audience=partner-api.
- 2 Build a token: header = {"alg":"HS256","typ":"JWT"}, payload = {"iss":"https://partner.example.com","aud":"partner-api","exp":(a future unix timestamp, e.g. now + 3600)}. Base64url-encode each part (no padding), join with dots, and append a third segment of any text as the (unverified) signature - this simulator checks the decoded claims, not the cryptographic signature.
- 3 Deploy to test.

### Verify:

```
GET /partner Header: Authorization: Bearer <your token>
-> VerifyJWT = success
```

Same call with the issuer changed to anything else in the payload  
-> 401 "JWT issuer is invalid"

## Exercise 6 - Custom fault for a malformed request

*Practices: RaiseFault, PreFlow ordering, fault short-circuiting*

**Your task:** On any proxy of your choice, add a RaiseFault policy as the very FIRST PreFlow step that always fires, returning HTTP 400 with body {"error":"exercise fault"}. Confirm every other attached policy is skipped.

### Guided solution

- 1 Open any proxy with at least 2 existing PreFlow policies (e.g. CustomerService-v1).
- 2 Add policy RaiseFault: statusCode=400, reasonPhrase=Bad Request, errorMessage={"error":"exercise fault"}.
- 3 It attaches to the end of PreFlow by default - if it isn't first, there's currently no PreFlow re-order control in the UI, so for this exercise just note this limitation and remove the OTHER PreFlow policies temporarily to isolate the effect, or accept that whichever policy runs before it still executes normally.

### Verify:

```
Any request to that proxy
-> RaiseFault = error, 400, response body {"error": "exercise fault"}
-> Any PreFlow policy listed AFTER RaiseFault shows "skipped due to earlier fault"
```

## Exercise 7 - A Product bundling two of your proxies, and a fresh App

*Practices: API Products spanning multiple proxies, Developer Apps, credential reuse*

**Your task:** Create an API Product called Partner-Bundle containing BOTH InventoryService-v1 (Exercise 1) and WarehouseService-v1 (Exercise 2). Create a new Developer App and associate it with Partner-Bundle. Use that app's key against both proxies.

### Guided solution

- 1 Products > + Create Product (or use the edit icon on an existing one) > name Partner-Bundle > add both proxies as chips.
- 2 Developers > pick or create a developer > Apps > + Create App > associate it with Partner-Bundle.
- 3 Copy the new app's Consumer Key.

### Verify:

```
GET /inventory Header: x-api-key: <new app's key> -> success
GET /warehouse Header: x-api-key: <same key> -> success (same credential, two different proxies)
```

## Exercise 8 - Two revisions, two different rate limits per environment

*Practices: revision cloning, independent per-environment behavior*

**Your task:** On InventoryService-v1 (Exercise 1), create revision 2 with SpikeArrest lowered to 2ps. Deploy revision 2 to dev only, keeping revision 1 live in test.

### Guided solution

- 1 Overview tab > "+ Create new revision".
- 2 On revision 2, open SpikeArrest and change rate to 2ps. Save.
- 3 Deploy tab > Deploy revision 2 to dev (test should already be on revision 1 from Exercise 1).

### Verify:

```
Environment=dev: 3 quick GET /inventory calls -> 3rd call gets 429
Environment=test: same 3 quick calls -> all succeed (still on the original 10ps revision)
```